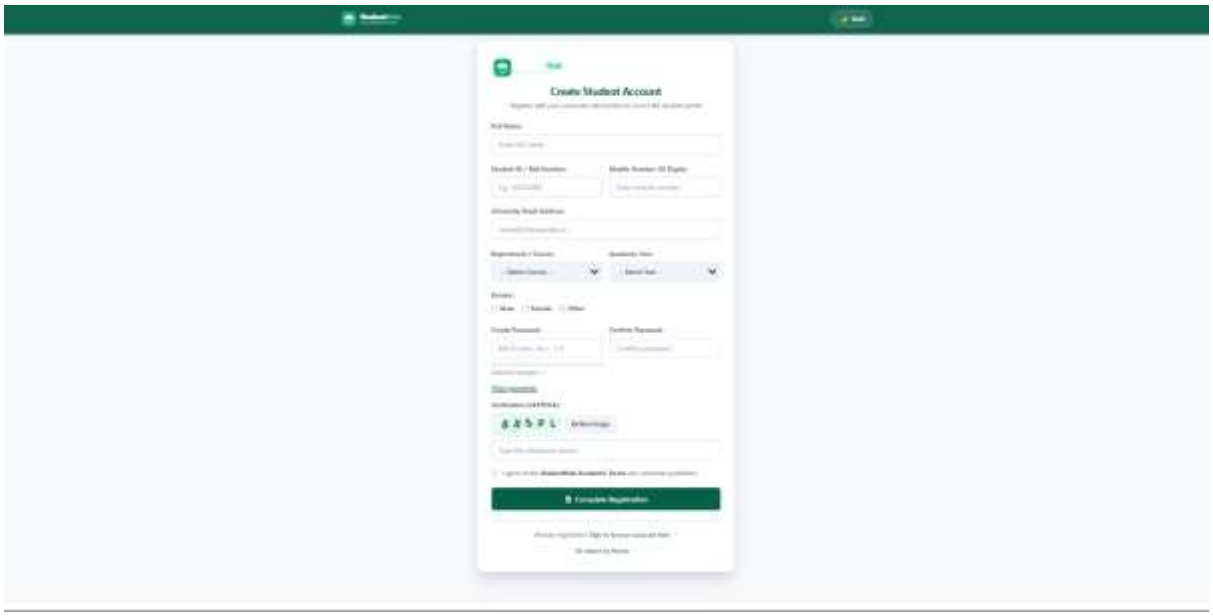
PRACTICAL-5

AIM: Build a registration form with HTML5 input types and attributes, add JavaScript validation (regex, inline error messages, password strength meter) and verify accessibility.

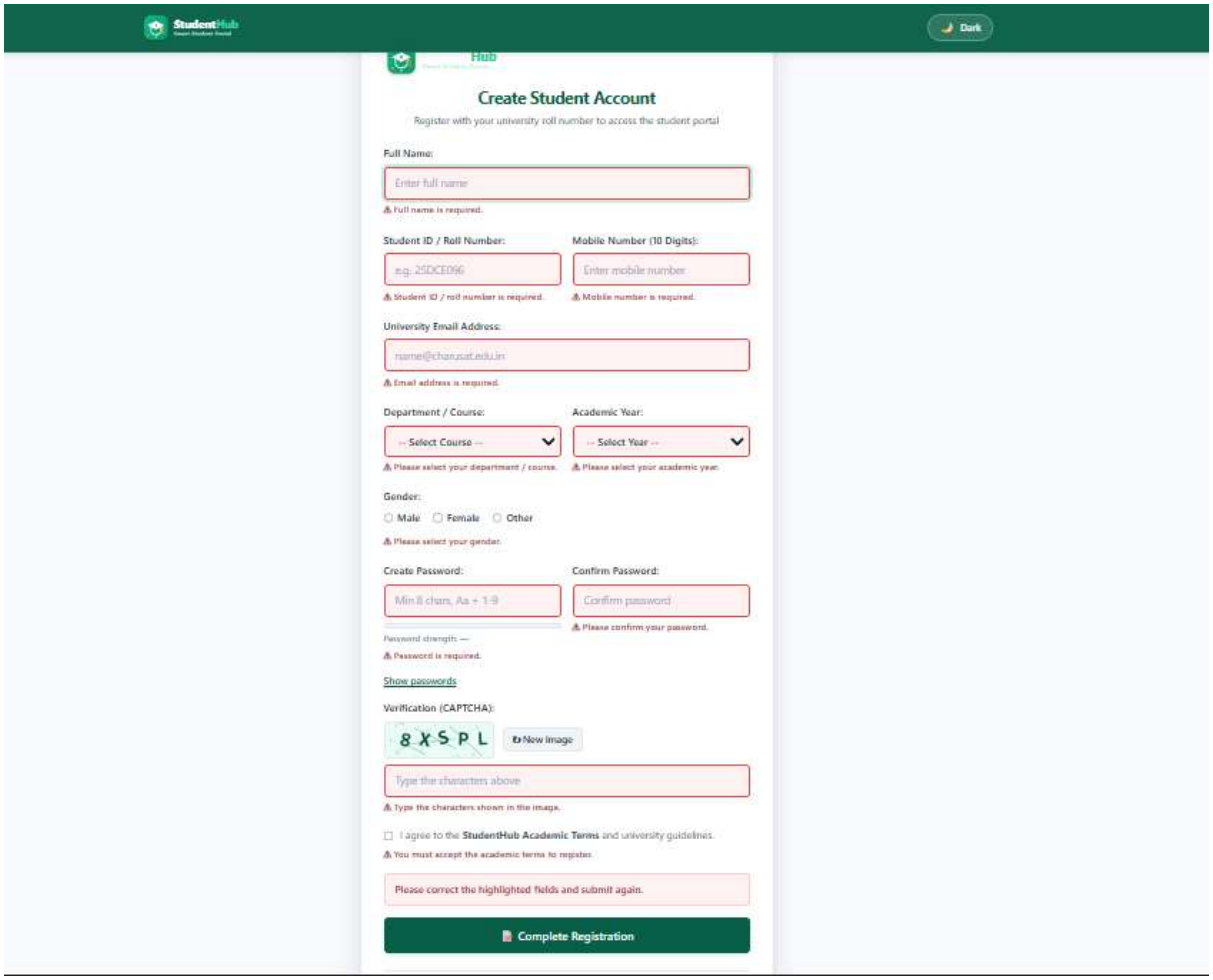
SOURCE CODE :

https://github.com/rachhadiyabrind-del/WDF_B/tree/main/Practical(1-4)(Studenthub)/javascript

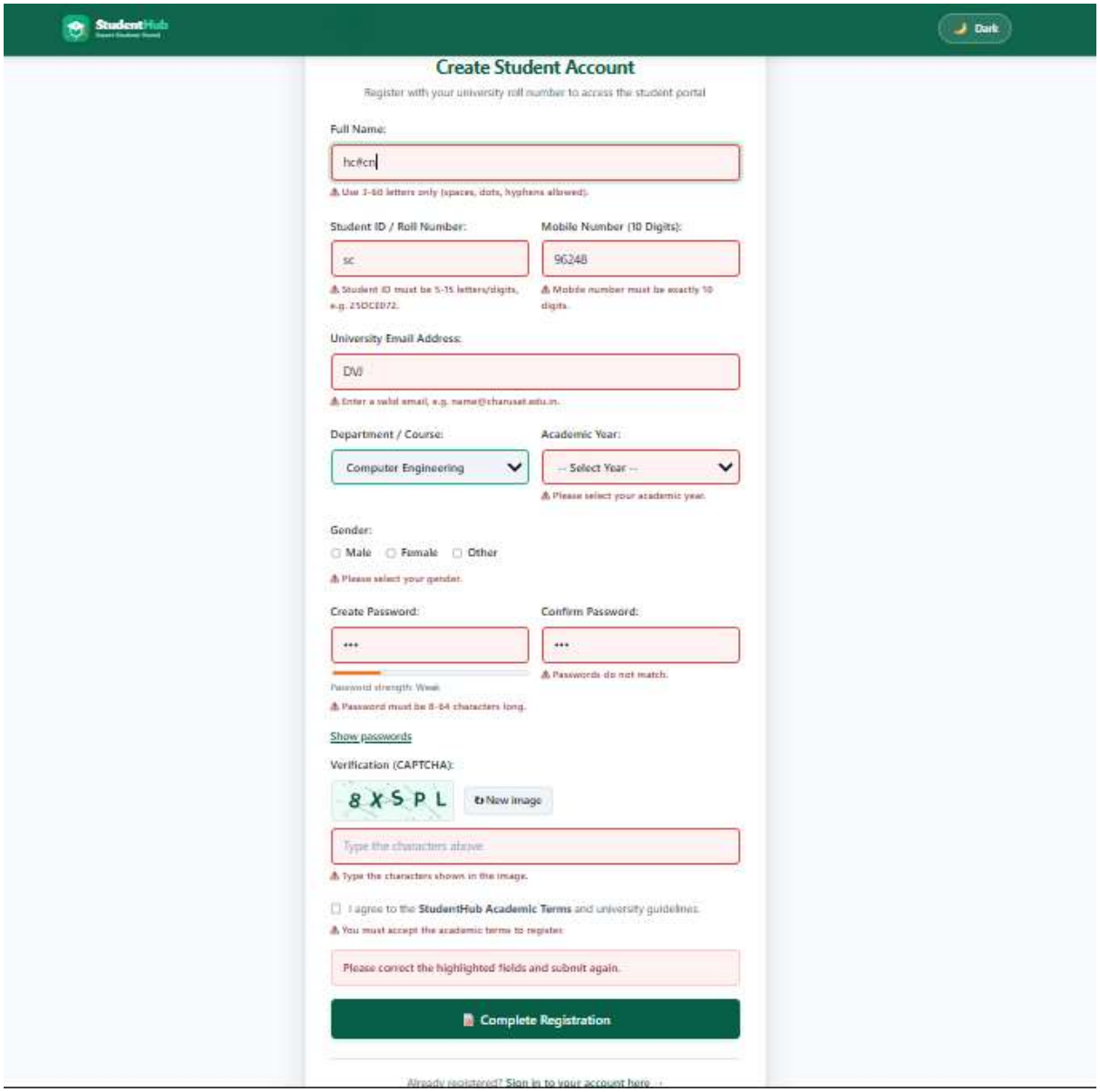
PREVIEW:



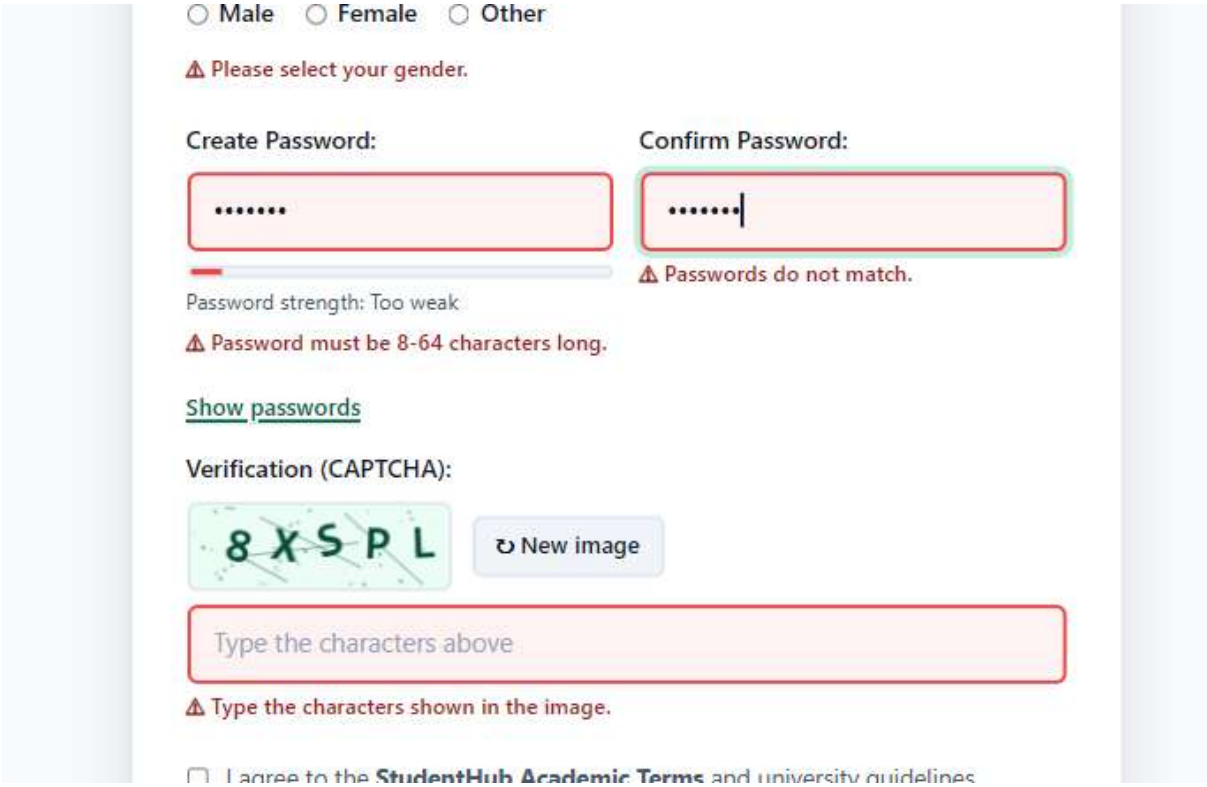
Registration form with HTML5 input types, labels and the Canvas CAPTCHA



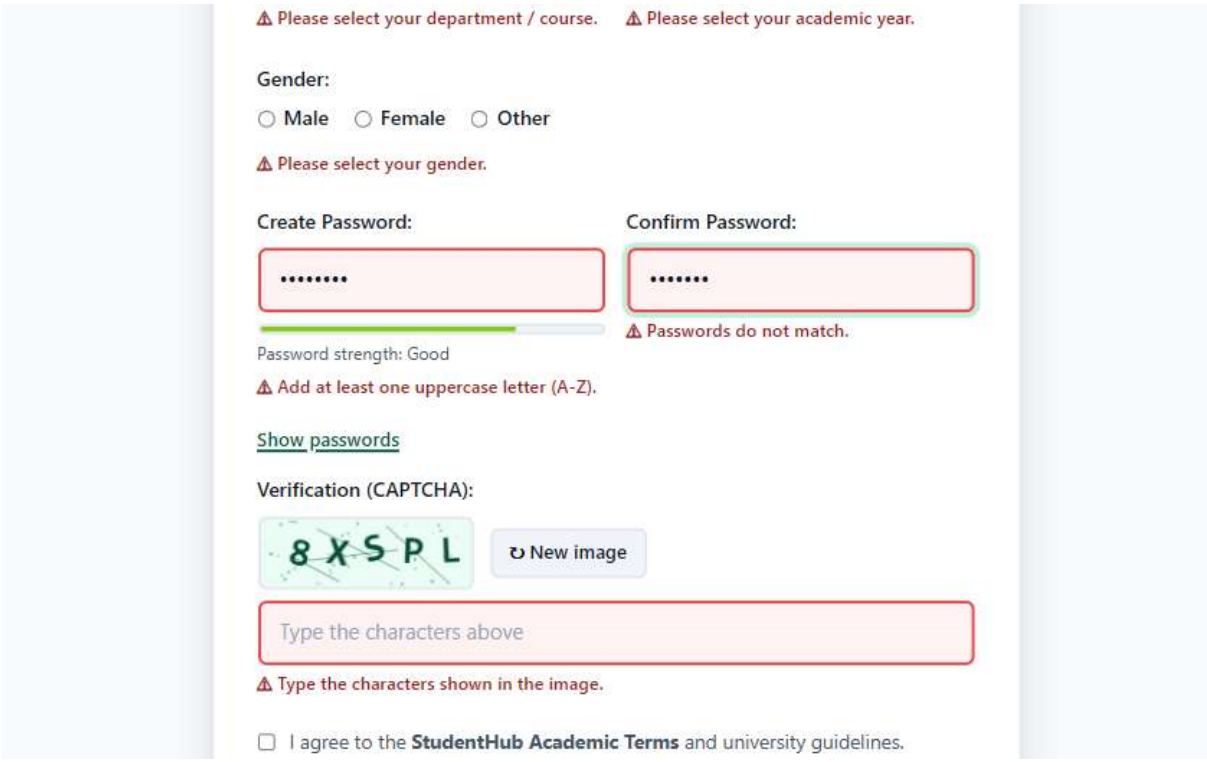
Submitting an empty form: an inline error appears next to every required field



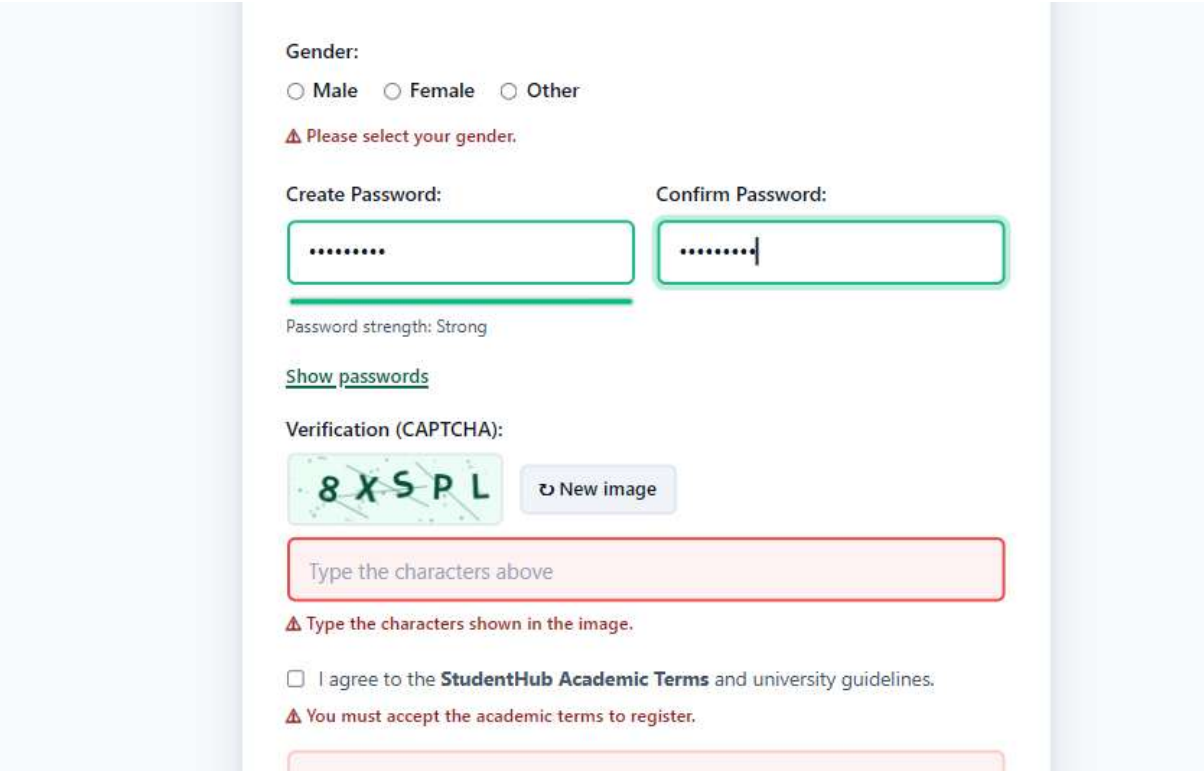
Invalid input: name with digits/symbols, short roll number, 5-digit mobile, bad e-mail, weak and mismatching passwords, wrong CAPTCHA



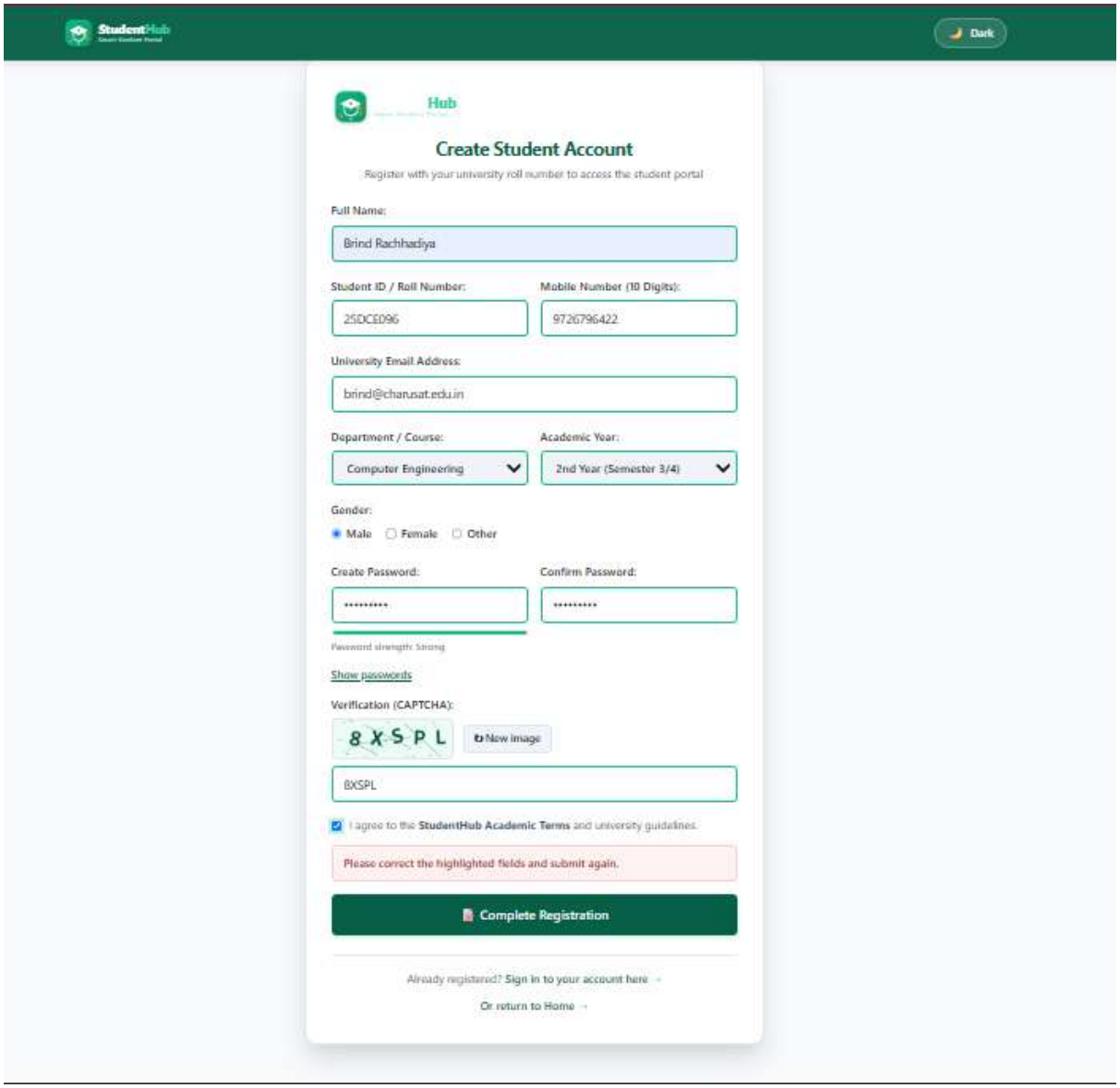
Password strength meter - "Too weak"



Password strength meter - medium



Password strength meter - strong



All fields valid: no error messages are shown

KEY QUESTIONS :

1. Are correct input types and attributes used?

Yes. The form uses type="email", type="tel" (with inputmode="numeric" and maxlength="10"), type="password", select lists, radio buttons and a checkbox. Attributes such as required, minlength, maxlength, autocomplete and aria-describedby are set, so mobile keyboards and password managers behave correctly.

```
<input type="tel" id="mobile" name="mobile" maxlength="10" inputmode="numeric"
autocomplete="tel" required aria-describedby="mobile-error">
<span class="field-error" id="mobile-error" role="alert"></span>
```

The browser's own bubbles are switched off with novalidate so that all messages look the same and come from our JavaScript; the server repeats every check (Practical 7 and 9), so JavaScript is only a convenience, never the only protection.

2. Are errors displayed near the fields?

Yes. Each field has its own <span id="<name>-error" role="alert"> directly below it. The validator writes the message there, adds the class has-error to the field (red border) and sets aria-invalid="true". When the field becomes valid the message is cleared and is-valid is added. Validation also runs in real time on input, change and blur (intermediate extension), so the user sees the problem as soon as they leave the field.

```
const PATTERNS = {
  fullname:  /^[A-Za-z][A-Za-z .'-]{2,59}$/,
  student_id: /^[A-Za-z0-9]{5,15}$/,
  mobile:    /^[0-9]{10}$/,
  email:     /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/
};
```

3. Is password strength checked?

Yes. The password must be 8-64 characters and contain an upper-case letter, a lower-case letter and a digit. In addition scorePassword() gives 0-4 points (length >= 8, length >= 12, mixed case, digit, special character) and the coloured bar under the field shows "Too weak / Weak / Good / Strong" while typing.

```
function scorePassword(value) {
  let score = 0;
  if (value.length >= 8) score++;
  if (value.length >= 12) score++;
  if (r.upper.test(value) && r.lower.test(value)) score++;
  if (r.digit.test(value)) score++;
  if (r.special.test(value)) score++;
  return Math.min(score, 4);
}
```

4. Is the form keyboard and screen-reader accessible?

- Every input has a visible <label for="..."> ; the radio group is wrapped with role="radiogroup" and aria-labelledby.
- Error spans use role="alert", so a screen reader announces a message when it appears; invalid fields get aria-invalid="true".
- The whole form can be completed with Tab / Shift+Tab / Space / Enter; the first invalid field receives focus after a failed submit; a visible focus outline is kept.
- The CAPTCHA canvas has role="img" and an aria-label, and a "New image" button; "Show passwords" is a real <button>.
- Colour is never the only signal: errors also have an icon and text.

5. Advanced extension - custom CAPTCHA on Canvas

A 5-character code (no confusing characters such as 0/O or 1/I) is drawn on a <canvas> with random rotation, noise lines and dots. The typed text is compared with the code in JavaScript. It is a demonstration only: a client-side CAPTCHA cannot stop bots, which is stated in the code comments; real protection is the server-side CSRF token and validation.